LAB 6 Report

Resilient Internet + Secure Microservices

CP352005 Network Programming | Khon Kaen University

0. สมาชิกกลุ่ม

ชื่อ-สกุล	รหัสนักศึกษา	Role
นายบุญปวีณ เรืองไพศาล	673380588-4	Network Architect, IaC/DevOps, QA/Test, Developer

1. Lab Objective

Lab 6 เป็นการอัพเกรด Lab 5 จาก “Working SME Network” ไปสู่ “Enterprise-grade Resilient WAN + Secured Microservices” โดยมีวัตถุประสงค์หลัก:

- Replace static routing ด้วย OSPF (FRRouting frr version 8.5)
- Add WAN failure detection ด้วย IP SLA shell monitor (ping 8.8.8.8 ทุก 5 วินาที)
- Implement edge security ด้วย iptables WAN-IN chain + MASQUERADE + static NAT
- Harden microservice exposure เปิดเฉพาะ port 8000 จาก Internet
- Structured logging format + X-Request-ID tracing ครบทุก service
- Automated tests: pytest compliance + integration (40+ test cases)

2. Network Topology

Lab 6 ใช้ physical layout เดียวกับ Lab 5 แต่เพิ่ม features ใหม่ด้วย Docker Compose + FRRouting:

- OSPF Area 0 ระหว่าง R1 (router-id 1.1.1.1) และ R2 (router-id 2.2.2.2)
- IP SLA shell monitor ใน r1_firewall.sh ตรวจ 8.8.8.8 และควบคุม default route ผ่าน vtysh
- iptables WAN-IN chain บน eth1 (ISP) ของ R1
- MASQUERADE สำหรับ LAN A + LAN B บน eth0 (Internet)
- Static NAT DNAT port 8000 → 192.168.10.10:8000

Address Table

Device	Interface	IP Address	Role
R1	eth0	10.255.0.2	Internet (NAT Edge)
R1	eth1	100.10.10.1	ISP WAN
R1	eth2	192.168.10.1	LAN A Gateway
R2	eth0	100.10.10.2	WAN

R2	eth1	192.168.20.1	LAN B Gateway
ServerA	eth0	192.168.10.10	Microservices A
ServerB	eth0	192.168.20.10	Microservices B
ClientA	eth0	192.168.10.50	Test Client A
ClientB	eth0	192.168.20.50	Test Client B

3. Infrastructure Deployment

Deploy ทั้งหมดด้วย Docker Compose:

```
$ docker compose down -v
$ docker compose up -d
```

ผลลัพธ์:

```
[+] Running 10/10
 Network networklab6_internet_net  Created
 Network networklab6_isp_net       Created
 Network networklab6_lan_a_net     Created
 Network networklab6_lan_b_net     Created
 Container R1      Started
 Container R2      Started
 Container ServerA Started
 Container ServerB Started
 Container ClientA Started
 Container ClientB Started
```

แสดงว่า network infrastructure ถูกสร้างสำเร็จ

4. Deploy Microservices

Microservices ถูก deploy อัตโนมัติผ่าน docker-compose.yml command บน ServerA และ ServerB:

```
$ docker exec ServerA ps
PID  USER  TIME  COMMAND
  1  root  0:00  sh -c pip install fastapi uvicorn ... && python start_services.py
 20  root  0:00  python upload_service.py      (port 8000)
 21  root  0:00  python processing_service.py  (port 8001)
 22  root  0:00  python ai_service.py          (port 8002)
 23  root  0:00  python gateway_service.py     (port 9000)
```

Port	Service	Lab 6 Upgrades
8000	Upload Service	Extension whitelist (.jpg .png .pdf .txt .docx), sanitize filename, X-Request-ID, X-Response-Time
8001	Processing Service	Validate SUPPORTED_OPERATIONS {thumbnail,resize,convert}, 400 on invalid, structured logging
8002	AI Service	Validate VALID_ANALYSIS_TYPES, request_id field in AIResponse body

9000	Gateway Service	CORS middleware, slowapi rate limit 20 req/min, path-traversal block, WorkflowResponse+request_id

5. Phase 1 — OSPF with FRRouting

R1 และ R2 ใช้ image frrouting/frr:latest และตั้งค่าผ่าน frr.conf โดย mount เข้า container:

5.1 config/r1/frr.conf (ไฟล์จริง)

```
frr version 8.5
frr defaults traditional
hostname R1
log syslog informational
service integrated-vtysh-config

interface eth0
 description Internet (NAT outside)
 ip address 10.255.0.2/24

interface eth1
 description ISP WAN to R2
 ip address 100.10.10.1/30

interface eth2
 description LAN A inside
 ip address 192.168.10.1/24

ip route 0.0.0.0/0 10.255.0.1

router ospf
 ospf router-id 1.1.1.1
 network 100.10.10.0/30 area 0
 network 192.168.10.0/24 area 0
 passive-interface eth2
 default-information originate

log timestamp precision 3
```

5.2 config/r2/frr.conf (ไฟล์จริง)

```
frr version 8.5
frr defaults traditional
hostname R2
log syslog informational
service integrated-vtysh-config

interface eth0
 description ISP WAN to R1
 ip address 100.10.10.2/30

interface eth1
 description LAN B inside
 ip address 192.168.20.1/24

router ospf
 ospf router-id 2.2.2.2
 network 100.10.10.0/30 area 0
 network 192.168.20.0/24 area 0
```

```
 passive-interface eth1

log timestamp precision 3
```

5.3 OSPF Verification

```
$ docker exec R1 vtysh -c 'show ip ospf neighbor'
Neighbor ID  Pri  State    Dead Time  Address       Interface
2.2.2.2        1  Full/DR  00:00:38   100.10.10.2   eth1

$ docker exec R2 vtysh -c 'show ip route ospf'
O   192.168.10.0/24 [110/20] via 100.10.10.1, eth0
O*E2 0.0.0.0/0 [110/1] via 100.10.10.1, eth0
```

OSPF neighbor state = FULL และ R2 เรียนรู้ default route อัตโนมัติจาก R1 โดยไม่ต้องตั้ง static route

6. Phase 2 — WAN Failure Detection (IP SLA Monitor)

IP SLA จำลองผ่าน background shell loop ใน scripts/r1_firewall.sh ที่ ping 8.8.8.8 ทุก 5 วินาที และใช้ vtysh เพิ่ม/ลบ default route:

```
# /usr/local/sbin/ip_sla_monitor.sh (สร้างและรันจาก r1_firewall.sh)
PROBE_TARGET="8.8.8.8"
INTERVAL=5
ROUTE_UP=1

while true; do
  if ping -c 1 -W 2 "$PROBE_TARGET" > /dev/null 2>&1; then
    if [ "$ROUTE_UP" -eq 0 ]; then
      vtysh -c "configure terminal" \
            -c "ip route 0.0.0.0/0 10.255.0.1" \
            -c "end" -c "write memory"
      ROUTE_UP=1
    fi
  else
    if [ "$ROUTE_UP" -eq 1 ]; then
      vtysh -c "configure terminal" \
            -c "no ip route 0.0.0.0/0 10.255.0.1" \
            -c "end" -c "write memory"
      ROUTE_UP=0
    fi
  fi
  sleep "$INTERVAL"
done &
```

6.1 IP SLA Verification

```
$ docker exec R1 ip link set eth0 down    # simulate ISP failure
$ docker exec R2 vtysh -c 'show ip route ospf'
(no output — default route removed by IP SLA monitor)

$ docker exec R1 ip link set eth0 up      # restore
$ docker exec R2 vtysh -c 'show ip route ospf'
O*E2 0.0.0.0/0 [110/1] via 100.10.10.1, eth0  (restored)
```

แสดงว่า IP SLA monitor ตรวจจับ Internet failure และลบ default route อัตโนมัติ และคืนค่าเมื่อ Internet กลับมา

7. Phase 3 — WAN Security (iptables ACL)

scripts/r1_firewall.sh เริ่มด้วยการ flush rules ทั้งหมด แล้วสร้าง WAN-IN chain:

```
# Flush existing rules
iptables -F
iptables -t nat -F
iptables -t mangle -F
iptables -X 2>/dev/null || true

# Default policies
iptables -P INPUT   ACCEPT
iptables -P FORWARD ACCEPT
iptables -P OUTPUT  ACCEPT

# Create WAN-IN chain
iptables -N WAN-IN
iptables -A WAN-IN -m state --state ESTABLISHED,RELATED -j ACCEPT
iptables -A WAN-IN -p tcp -s 192.168.20.0/24 -d 192.168.10.0/24 -j ACCEPT
iptables -A WAN-IN -p 89 -j ACCEPT   # allow OSPF
iptables -A WAN-IN -j LOG --log-prefix '[WAN-IN BLOCK] ' --log-level 4
iptables -A WAN-IN -j DROP
iptables -A FORWARD -i eth1 -j WAN-IN
```

7.1 ACL Verification

```
$ docker exec R1 iptables -L WAN-IN -n -v
Chain WAN-IN (1 references)
 pkts bytes target  prot  source             destination
   14  1680 ACCEPT  all   0.0.0.0/0          0.0.0.0/0   state RELATED,ESTABLISHED
   12  1440 ACCEPT  tcp   192.168.20.0/24    192.168.10.0/24
    1    76 ACCEPT  89    0.0.0.0/0          0.0.0.0/0
    3   180 LOG     all   0.0.0.0/0          0.0.0.0/0
    3   180 DROP    all   0.0.0.0/0          0.0.0.0/0
```

เฉพาะ LAN B → LAN A TCP และ OSPF ถูกอนุญาต traffic อื่นถูก log และ DROP

8. Phase 4 — Microservice Exposure Control

```
# MASQUERADE outbound for LAN A and LAN B
iptables -t nat -A POSTROUTING -s 192.168.10.0/24 -o eth0 -j MASQUERADE
iptables -t nat -A POSTROUTING -s 192.168.20.0/24 -o eth0 -j MASQUERADE

# Static NAT: expose only port 8000 (Upload Service) to Internet
iptables -t nat -A PREROUTING -i eth0 -p tcp --dport 8000 \
  -j DNAT --to-destination 192.168.10.10:8000

# Allow forwarding for DNATed traffic only
iptables -A FORWARD -i eth0 -p tcp -d 192.168.10.10 --dport 8000 \
  -m state --state NEW,ESTABLISHED,RELATED -j ACCEPT

# Block all other ports from Internet to LAN
iptables -A FORWARD -i eth0 -d 192.168.10.0/24 -j DROP
iptables -A FORWARD -i eth0 -d 192.168.20.0/24 -j DROP
```

8.1 Exposure Verification

```
# Internet -> port 8000 (allowed via static NAT)
$ curl http://10.255.0.2:8000/health
{"status":"healthy","service":"upload-service","version":"2.0.0"}

# Internet -> port 9000 (blocked)
$ curl http://10.255.0.2:9000/stats
curl: (7) Failed to connect to 10.255.0.2 port 9000: Connection refused

# Internet -> port 8001 (blocked)
$ curl http://10.255.0.2:8001/health
curl: (7) Failed to connect to 10.255.0.2 port 8001: Connection refused
```

9. Phase 5 — Observability Foundation

ทุก service ใช้ logging format เดียวกันจาก basicConfig ใน Python:

```
# Log format (ทุก service)
format='%(asctime)s.%(msecs)03d [%(levelname)s] %(name)s | req=%(request_id)s |
%(message)s'
datefmt='%Y-%m-%dT%H:%M:%S'
```

ตัวอย่าง log output จริงเมื่อรัน workflow:

```
2026-03-30T10:15:22.301 [INFO] gateway | req=gw-trace-42 | POST /process-file -> 200
(312.4ms)
2026-03-30T10:15:22.450 [INFO] gateway | req=gw-trace-42 | Workflow start:
wf_20260330_101522_gw-trace-42
2026-03-30T10:15:22.601 [INFO] upload  | req=gw-trace-42 | Uploaded: abc123
name=test.jpg size=36
2026-03-30T10:15:25.112 [INFO] process | req=gw-trace-42 | Processing done: abc123 in
2.51s
2026-03-30T10:15:26.880 [INFO] ai      | req=gw-trace-42 | AI analysis done: abc123
confidence=0.87
2026-03-30T10:15:26.920 [INFO] gateway | req=gw-trace-42 | Workflow done:
wf_20260330_101522_gw-trace-42 in 4.47s
```

X-Request-ID ถูกส่งต่อจาก gateway ไปยัง upload, processing, AI ทุก call ผ่าน header X-Request-ID ทำให้ trace ได้ครบทั้ง pipeline

10. Phase 6 — Network Validation

10.1 Test 1 — ClientA → ServerB via OSPF

```
$ docker exec ClientA curl http://192.168.20.10:8000/health
{"status":"healthy","service":"upload-service","version":"2.0.0","timestamp":"..."}

$ docker exec ClientA ping -c 3 192.168.20.10
3 packets transmitted, 3 received, 0% packet loss
```

10.2 Test 2 — R2 loses default if ISP unreachable

```
$ docker exec R1 ip link set eth0 down
$ docker exec R2 vtysh -c 'show ip route ospf'
```

```
(no default route — removed by IP SLA monitor)
```

10.3 Test 3 — Internet cannot access backend ports

```
$ curl http://10.255.0.2:8001/health   # blocked
curl: (7) Connection refused
$ curl http://10.255.0.2:8002/health   # blocked
curl: (7) Connection refused
$ curl http://10.255.0.2:9000/stats    # blocked
curl: (7) Connection refused
```

10.4 Test 4 — Gateway reachable from Internet

```
$ curl http://10.255.0.2:8000/health   # allowed (static NAT)
{"status":"healthy","service":"upload-service","version":"2.0.0"}
```

11. Test Plan Summary

Test	Expected Result	Status
OSPF neighbor (vtysh show ip ospf neighbor)	FULL state	PASS
show ip route ospf on R2	Remote LAN + default visible	PASS
ISP unplug (ip link set eth0 down)	Default route disappears	PASS
Internet → port 8000	HTTP 200 OK	PASS
Internet → port 9000 / 8001 / 8002	Connection refused (blocked)	PASS
LAN A ↔ LAN B ping	0% packet loss	PASS

pytest ใช้ test_lab6_compliance.py (7 classes) และ test_lab6_integration.py (6 classes) รวม 40+ test cases ครอบคลุมที่ PASS ทั้งหมด

12. Project File Overview

File	Purpose
docker-compose.yml	Deploy R1 R2 ServerA ServerB ClientA ClientB + 4 Docker networks
config/r1/frr.conf	FRR 8.5: OSPF router-id 1.1.1.1, interface IPs, default-information originate
config/r1/daemons	Enable zebra + ospfd + bfdd
config/r2/frr.conf	FRR 8.5: OSPF router-id 2.2.2.2, interface IPs, learns default from R1
config/r2/daemons	Enable zebra + ospfd + bfdd
scripts/r1_firewall.sh	iptables flush + WAN-IN chain + MASQUERADE + static NAT + IP SLA monitor
gateway_service.py	CORS, slowapi 20 req/min, X-Request-ID middleware, path-traversal block
upload_service.py	Extension whitelist, filename sanitize, X-Request-ID, MAX_FILE_SIZE_MB

processing_service.py	SUPPORTED_OPERATIONS validation, structured logging, request ID
ai_service.py	VALID_ANALYSIS_TYPES validation, request_id in AIResponse
start_services.py	Exponential backoff health retry, OSPF vtysh check, SIGTERM handler
test_lab6_compliance.py	40+ pytest cases: health, upload, processing, AI, gateway, security, concurrency
test_lab6_integration.py	Integration: lifecycle, request tracing, batch, error handling, performance
.env.example	All env vars: ports, URLs, ALLOWED_ORIGINS, MAX_FILE_SIZE_MB, FRR_ENABLED
requirements.txt	fastapi 0.111, uvicorn 0.29, httpx 0.27, slowapi 0.1.9, pytest 8.2 (pinned)

13. Architectural Classification

Feature	Implementation
Routed WAN with dynamic routing	OSPF Area 0 via FRRouting 8.5
Conditional default injection	default-information originate (R1 frr.conf)
WAN ACL firewall	iptables WAN-IN chain on eth1 (r1_firewall.sh)
NAT overload	MASQUERADE LAN A+B via eth0 (r1_firewall.sh)
Controlled service publishing	DNAT port 8000 only, DROP 8001/8002/9000 (r1_firewall.sh)
WAN failure detection	IP SLA shell monitor: ping 8.8.8.8 + vtysh add/remove route
Observability	Structured logging + X-Request-ID tracing ทุก service
Rate limiting	slowapi 20 req/min on /process-file, 30/min on /upload-only
Input validation	Extension whitelist, filename sanitize, operation/type validation
Automated testing	pytest 40+ cases: compliance + integration + security + concurrency

14. Readiness Score After LAB 6

Category	Score
Routing	9 / 10
WAN Intelligence	9 / 10
Security	6 / 10
Microservice Exposure	8 / 10
Scalability	8 / 10
Observability	8 / 10
Overall	8.5 / 10 — Enterprise Branch-Ready

15. Conclusion

Lab 6 ประสบความสำเร็จในการยกระดับสถาปัตยกรรมเครือข่ายเป็น Enterprise Branch ผลลัพธ์สำคัญ:

- OSPF (FRRouting 8.5) ทำงานถูกต้อง neighbor = FULL, R2 เรียนรู้ default route อัตโนมัติ
- IP SLA shell monitor ตรวจจับ ISP failure และควบคุม default route ผ่าน vtysh อัตโนมัติ
- iptables WAN-IN chain บล็อค unsolicited traffic อนุญาตเฉพาะ LAN B → LAN A TCP และ OSPF
- Static NAT DNAT port 8000 เท่านั้นที่เข้าถึงได้จาก Internet port 8001/8002/9000 ยังเป็น private
- Structured logging + X-Request-ID tracing ครบทั้ง pipeline ตั้งแต่ gateway ถึง AI service
- Automated tests 40+ cases (compliance + integration + security + concurrency) ผ่าน ทั้งหมด

Lab นี้เป็นการผสมผสาน Dynamic Routing, WAN Resilience, Edge Security, Controlled Service Exposure และ Observability เข้าด้วยกันผ่าน 15 ไฟล์ที่ทำงานจริงทั้งหมด